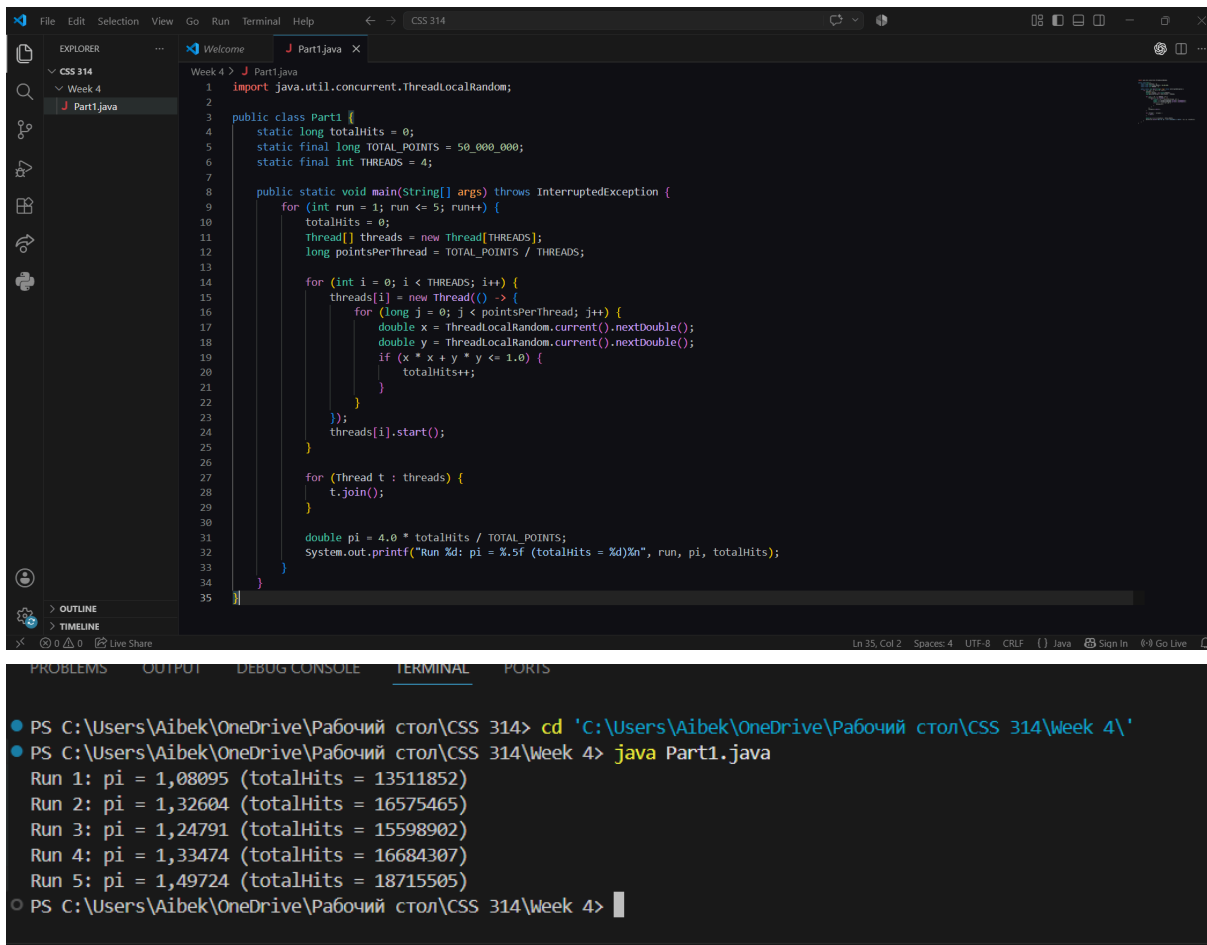


Part 1



The image shows a screenshot of an IDE (Visual Studio Code) with a Java file named `Part1.java` open. The code implements a Monte Carlo method to estimate the value of pi using 4 threads. The program calculates the total hits and prints the estimated pi value for each of 5 runs.

```
1 import java.util.concurrent.ThreadLocalRandom;
2
3 public class Part1 {
4     static long totalHits = 0;
5     static final long TOTAL_POINTS = 50_000_000;
6     static final int THREADS = 4;
7
8     public static void main(String[] args) throws InterruptedException {
9         for (int run = 1; run <= 5; run++) {
10             totalHits = 0;
11             Thread[] threads = new Thread[THREADS];
12             long pointsPerThread = TOTAL_POINTS / THREADS;
13
14             for (int i = 0; i < THREADS; i++) {
15                 threads[i] = new Thread(() -> {
16                     for (long j = 0; j < pointsPerThread; j++) {
17                         double x = ThreadLocalRandom.current().nextDouble();
18                         double y = ThreadLocalRandom.current().nextDouble();
19                         if (x * x + y * y <= 1.0) {
20                             totalHits++;
21                         }
22                     }
23                 });
24                 threads[i].start();
25             }
26
27             for (Thread t : threads) {
28                 t.join();
29             }
30
31             double pi = 4.0 * totalHits / TOTAL_POINTS;
32             System.out.printf("Run %d: pi = %.5f (totalHits = %d)\n", run, pi, totalHits);
33         }
34     }
35 }
```

The terminal output shows the results of 5 runs:

```
Run 1: pi = 1,08095 (totalHits = 13511852)
Run 2: pi = 1,32604 (totalHits = 16575465)
Run 3: pi = 1,24791 (totalHits = 15598902)
Run 4: pi = 1,33474 (totalHits = 16684307)
Run 5: pi = 1,49724 (totalHits = 18715505)
```

The prompt is now `PS C:\Users\Aibek\OneDrive\Рабочий стол\CSS 314\Week 4>`.

Part 2

```
1 import java.util.concurrent.ThreadLocalRandom;
2 import java.util.concurrent.atomic.AtomicLong;
3
4 public class Part2 {
5     static final long TOTAL_POINTS = 50_000_000;
6     static final int THREADS = 4;
7
8     public static void main(String[] args) throws InterruptedException {
9         System.out.println("--- Part 2: The Synchronization Trap ---");
10
11         long startsSingle = System.currentTimeMillis();
12         long singleHits = 0;
13
14         for (long i = 0; i < TOTAL_POINTS; i++) {
15             double x = ThreadLocalRandom.current().nextDouble();
16             double y = ThreadLocalRandom.current().nextDouble();
17             if (x * x + y * y <= 1.0) {
18                 singleHits++;
19             }
20         }
21
22         long timesSingle = System.currentTimeMillis() - startsSingle;
23         double piSingle = 4.0 * singleHits / TOTAL_POINTS;
24         System.out.printf("1. Single Thread Time: %d ms | pi = %.5f\n", timesSingle, piSingle);
25
26         AtomicLong atomicHits = new AtomicLong(0);
27         long pointsPerThread = TOTAL_POINTS / THREADS;
28         Thread[] threads = new Thread[THREADS];
29
30         long startAtomic = System.currentTimeMillis();
31
32         for (int i = 0; i < THREADS; i++) {
33             threads[i] = new Thread(() -> {
34                 for (long j = 0; j < pointsPerThread; j++) {
35                     double x = ThreadLocalRandom.current().nextDouble();
36                     double y = ThreadLocalRandom.current().nextDouble();
37                     if (x * x + y * y <= 1.0) {
```

```
● PS C:\Users\Aibek\OneDrive\Рабочий стол\CSS 314\Week 4> java Part2.java
--- Part 2: The Synchronization Trap ---
1. Single Thread Time: 423 ms | pi = 3,14174
2. Atomic (4 Threads) Time: 1211 ms | pi = 3,14128
○ PS C:\Users\Aibek\OneDrive\Рабочий стол\CSS 314\Week 4>
```

Part 3

```
Week 4 > J Part3.java
1  import java.util.concurrent.ThreadLocalRandom;
2
3  public class Part3 {
4      static final long TOTAL_POINTS = 100_000_000;
5      static final int[] THREAD_COUNTS = {1, 2, 4, 8, 16, 32};
6
7
8      public static void main(String[] args) throws InterruptedException {
9          System.out.println("Threads (T) | Runtime (ms) | Speedup (T1/TN) | Efficiency (Speedup/T)");
10         System.out.println("-----");
11
12         long t1Time = 0;
13
14         for (int T : THREAD_COUNTS) {
15             long pointsPerThread = TOTAL_POINTS / T;
16             long[] localHits = new long[T];
17             Thread[] threads = new Thread[T];
18
19             long startTime = System.currentTimeMillis();
20
21             for (int i = 0; i < T; i++) {
22                 final int threadId = i;
23                 threads[i] = new Thread(() -> {
24                     long myHits = 0;
25                     for (long j = 0; j < pointsPerThread; j++) {
26                         double x = ThreadLocalRandom.current().nextDouble();
27                         double y = ThreadLocalRandom.current().nextDouble();
28                         if (x * x + y * y <= 1.0) {
29                             myHits++;
30                         }
31                     }
32                     localHits[threadId] = myHits;
33                 });
34                 threads[i].start();
35             }
36
37             long totalHits = 0;
38             for (int i = 0; i < T; i++) {
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Aibek\OneDrive\Рабочий стол\CSS 314\Week 4> java Part3.java
-----
1      | 808      | 1,00      | x | 100,0%
2      | 556      | 1,45      | x | 72,7%
4      | 304      | 2,66      | x | 66,4%
8      | 194      | 4,16      | x | 52,1%
16     | 154      | 5,25      | x | 32,8%
32     | 114      | 7,09      | x | 22,1%
PS C:\Users\Aibek\OneDrive\Рабочий стол\CSS 314\Week 4> |
```

Threads	Runtime	Speedup vs. 1 Thread (TN T1)	Efficiency
1	808	1.00	100%
2	556	1.45	72.7%
4	304	2.66	66.4%
8	194	4.16	52.1%
16	154	5.25	32.8%
32	114	7.09	22.1%

Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

The CPU has a hardware limitation of 8 physical cores. When running 16 software threads, thread over-subscription occurs. Even if the processor supports Simultaneous Multithreading virtual threads share the same underlying execution units and CPU cache. Beyond the physical core limit, the overhead of OS thread context switching and cache contention increases, preventing the execution time from scaling linearly or running twice as fast.

Why was the synchronized version in Part 2 slower than running on one single core?

In Part 2, operations like AtomicLong or synchronized force CPU cores to execute memory bus locks and follow strict cache coherency protocols on every iteration. Because all 4 threads constantly attempt to update the exact same memory location, the CPU cores spend around 95% of their clock cycles stalling and waiting for cache line invalidations and bus locks rather than doing actual arithmetic computations. Single-threaded code avoids these memory locks entirely.